

Executive Summary

This document defines a complete requirements and design package for the **NIC College Ambassador (NIC CAP)** backend. It covers project goals, scope, and detailed system design (use cases, data models, architecture, security, testing, deployment, etc.) to guide all stakeholders (product owners, developers, QA, DevOps). The primary goal is to build a **production-ready** Django/DRF application (using Cookiecutter Django and Clean Architecture) to manage student applications, ambassadors, and campus activities. Key features include a public site (Home/About/CAP info), an online application form, secured Ambassador accounts (Google OAuth), a personal Ambassador dashboard (profile, activity reports), and an Admin/Content management interface. Non-functional requirements (performance, security, scalability, etc.) are specified. We adopt industry best-practices: *clean architecture* (separating domain, application, infrastructure layers [29†L61-L69] [29†L89-L98]), *role-based access control* (RBAC) with clear personas [4†L126-L135] , containerized deployment (Docker, Gunicorn, Nginx), Celery for background tasks, and AWS services (RDS, S3) for reliability. All API endpoints are fully specified (OpenAPI YAML), and extensive documentation, diagrams (use-case, ER, state machines, flows), and CI/CD/runbook instructions are provided. Visual aids (diagrams, tables) are used liberally to ensure cross-team clarity [42†L740-L743] . This package ensures everyone shares the same understanding of what to build and how to operate it [42†L774-L777] , reducing ambiguity and enabling effective development and deployment.

1. Project Overview, Goals, Scope, NFRs, Constraints, Assumptions

- **Goals:** Implement NIC CAP backend to quickly onboard college ambassadors and manage campus activities. Enable students to apply online and track status; allow approved students (Ambassadors) to report activities; allow NIC staff to manage content and approve ambassadors. Use Django/DRF with a modern, maintainable code structure (Clean Architecture, Cookiecutter template [29†L89-L98] [24†L48-L51]). Emphasize security, scalability, and ease of future expansion.
- **Scope (MVP):** Public website with Home/About CAP pages, online *Application Form* (fields: name, college, email, motivation, etc.), secured *Ambassador Login/Registration* (via Google OAuth), an Ambassador Dashboard (view/update profile, submit activity reports), and an Admin/Content panel (approve applications, manage ambassadors, post news/events). Core data entities include Users, Applications, Ambassadors, Activity Reports, and Events. Excluded (Phase 1): gamification, mobile apps, complex analytics (can be future phases).
- **Non-Functional Requirements:**
 - *Performance:* Pages should load rapidly (ideally <3s), efficient database queries, use caching for static content. Employ pagination on large lists.
 - *Scalability:* Design for growth (e.g. multiple Gunicorn workers, DB replica if needed). Use AWS RDS for managed Postgres, AWS ElastiCache (Redis) for caching/broker.
 - *Security:* All traffic over HTTPS (TLS); enforce HTTPS via Cloudflare/Nginx (Cookiecutter notes “HTTPS is on by default” in Docker deployments [54†L94-L100]). Encrypt sensitive data **in transit** (TLS) and **at rest** (RDS, S3 with SSE) [19†L19-L27] [19†L34-L42] . CSRF protection and XSS/SQL injection guards via Django's defaults.
 - *Mobile-Responsive:* UI must work on phones/tablets (critical, as many students use mobile browsers).

- **Availability:** Target high uptime (assume SLA TBD; e.g. 99.9%); design for easy restart and failover. Use Docker health checks and auto-restart policies.
- **Data Protection & Compliance:** Treat personal data (student info, emails) as sensitive. Use AWS Shared Responsibility guidelines (e.g. GDPR-like controls [19†L59-L63]). Implement data retention policies (e.g. remove stale applications after some period). Encrypt secrets at rest (KMS) and in transit.
- **Logging & Monitoring:** Log all significant events (login, approvals, errors). Integrate Sentry or similar for error tracking. Set up health monitoring (e.g. via Prometheus/CloudWatch) and alerts on failures.
- **Constraints:** Use Django (latest stable, e.g. 4.x), Python 3.11+, PostgreSQL, Django REST Framework (DRF), and follow a *cookiecutter-django* project template [24†L48-L51] . Deployment on Docker (gunicorn, nginx), AWS cloud (EC2/ECS, RDS, S3), Cloudflare CDN for SSL. Time constraint ~8–12 weeks to MVP. Use Clean Architecture principles (separate domain logic) [29†L61-L69] [29†L89-L98] .
- **Assumptions:** Unspecified items are noted explicitly. For example, expected user volume is unknown (assume moderate — perhaps hundreds of applications). No paid tiers or premium features at launch. Exact legal requirements not provided (assume general data privacy best practices). NIC will provide domain & hosting environment. Email service (e.g. SES or SMTP) will be available. If not specified, assume default values (e.g. image upload limits, session timeouts can be configured later).

2. User Personas & Roles (RBAC)

We define clear *personas* (user roles) with associated permissions [4†L126-L135] . Each persona is an archetype with specific access rights. We implement role-based access control (RBAC) in Django (e.g. via groups/permissions) keyed to these roles.

Role/Persona	Description	Key Permissions/Actions
Guest (Public)	Unauthenticated user browsing the site.	View public pages (Home, About CAP, Events/News lists), read FAQs, <i>start</i> application (can view form, but must register to submit).
Applicant	Prospective ambassador (student) before approval.	Create account (via Google OAuth), submit Application form, view own application status/updates.
Ambassador	Approved ambassador (student user).	(All Applicant rights) plus: edit personal Profile, access Ambassador Dashboard, submit new Activity Reports, view past reports, browse shared resources.
Content Editor	NIC staff responsible for website content.	Login to admin/CMS, create/edit static content (news, events, FAQs), manage site pages.
Admin (NIC Admin)	NIC program administrator.	All permissions of Content Editor plus: review/list Applications, Approve/Reject applications, activate Ambassador accounts, manage Ambassador accounts, view system analytics or logs.
External System	(Not a “person” but external actor)	<i>Google OAuth</i> (performs user authentication), <i>Email Service</i> (sends notifications).

Notes: Using personas ensures granular control [4†L126-L135] . For example, every ambassador account is linked to a “User” with role=Ambassador; only Admins can change application status. We will map roles to Django groups/permissions: e.g. group “Ambassador” has can_view_dashboard=True, can_submit_report=True, etc.

3. Use Case Diagram & Textual Use Cases

【46†embed_image】 *Figure: Example UML Use Case diagram illustrating actors (Guest/Applicant, Ambassador, Admin, Content Editor) and their interactions with system functions.* This high-level diagram shows each actor’s main capabilities.

- **Guest:** Browse CAP site, read about program, view events/news. Initiate application process by clicking “Apply.”
- **Applicant:** (Authenticated Student) Fill out and submit the application form with personal/college details and motivation. Receive confirmation. Track application status via portal or email.
- **Ambassador:** (Authenticated, Approved) Log in via Google OAuth. View personal dashboard: update profile (college, contact info), view application/ambassador status. Submit activity reports (title, description, date, supporting files). View past submitted reports and status. Access any shared resources or community announcements.
- **Admin:** Log into admin panel. View list of submitted Applications. Open an Application case to view details, then Approve or Reject. If approved, create/activate Ambassador account. Manage Ambassador users (deactivate if needed). Manage site content: add/edit Events, News, FAQs. Monitor system (view metrics or logs).
- **Content Editor:** Log into CMS/editor interface. Create or update public content such as News articles, Upcoming Events, FAQs, static pages. Publish changes.

Each use-case is further detailed in the SRS. For example, “*Submit Application*”: Actor = Applicant, Precondition = user has opened site and clicked Apply. Description = “Applicant fills the online form (name, college, email, motivation) and submits it. System validates input, creates an Application record, and sends an acknowledgment email.” Postcondition = Application status = Pending Review. (We would provide similar text use-case templates for all major actions.)

4. User Journeys / Flows

Below are typical user flows for each persona, illustrated with Mermaid flowcharts:

Applicant/Guest Flow:

```

flowchart LR
    A[Visit NIC CAP Website] --> B[Read Home/About];
    B --> C[Click "Apply Now"];
    C --> D[Fill Application Form];
    D --> E{Submit Application?};
    E -->|Yes| F[Form submitted to backend];
    F --> G[Show confirmation message];
    G --> H[Admin reviews application (offline)];
  
```

```

H --> I{Decision};
I -->|Approve| J[Account activated, Ambassador created];
I -->|Reject| K[Rejection email sent];
...

```

Step Description: Guest reads site info then initiates application. The form is submitted (via `/api/v1/applications`). After submission, the system stores the application and notifies Admin. Admin later reviews it and either approves (creating an Ambassador account and sending a welcome email) or rejects (sending a rejection email).

****Ambassador Flow:****

```

```mermaid

```

flowchart LR

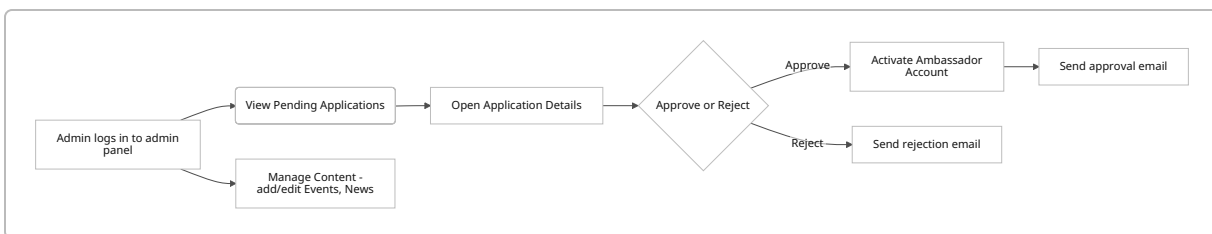
```

A[Ambassador logs in via Google OAuth] --> B[View Dashboard];
B --> C[Update Profile (college, contact info)];
B --> D[Submit New Activity Report];
D --> E[Fill report details -> Save];
E --> F[Report sent for review];
B --> G[View Past Reports & Status];
B --> H[View News/Events];

```

Ambassadors authenticate and land on their personal dashboard. They can edit their profile, submit a new activity report (which goes to Pending Review), and view history of submitted reports. They also see any relevant news or event listings.

### Admin Flow:



Admins use a secure dashboard. They review new applications (with ability to view, approve or reject). Approving triggers account creation and email; rejecting notifies the applicant. Admins also navigate to content management pages to update public site content.

### Content Editor Flow:

flowchart LR

```

A[Editor logs into CMS] --> B[Create/Edit content items (News, FAQs, etc.)];
B --> C[Publish updates to site];

```

Editors log into the content management interface (could be Django admin or a custom CMS) and update static content like FAQ pages, news items, or event listings. Changes go live immediately after saving.

## 5. High-Level System Architecture

```
graph LR
 Browser((User Browser)) -->|HTTPS| Cloudflare[Cloudflare CDN & SSL];
 Cloudflare --> Nginx[Nginx Reverse Proxy];
 Nginx --> Gunicorn[Gunicorn WSGI Server];
 Gunicorn --> DjangoApp[Django Backend (DRF API)];
 DjangoApp --> Postgres[(PostgreSQL)];
 DjangoApp --> Redis[(Redis Cache & Broker)];
 Redis --> Celery[Celery Workers];
 DjangoApp --> S3[(AWS S3 for Media/Static)];
 DjangoApp --> Sentry[(Sentry Logging)];
 DjangoApp --> EmailSvc[Email Service];
 GoogleOAuth[(Google OAuth)] --> DjangoApp;
 EmailSvc -. UserMailbox((User Email)) .->
```

*Figure: High-level deployment architecture.* Users access the site through **Cloudflare** (providing SSL and CDN). Nginx acts as a reverse-proxy and load balancer, forwarding requests to **Gunicorn**, which runs the Django application (DRF). The application uses **PostgreSQL** for persistent data and **Redis** as cache and Celery broker; **Celery workers** handle asynchronous tasks (e.g. sending emails). Media and static files are stored in **AWS S3** (with server-side encryption) [19†L34-L42]. Errors are sent to **Sentry**. **Google OAuth** is used for user authentication. An external Email Service (e.g. AWS SES) sends notifications to user inboxes.

### Components:

- **Django + DRF API:** Business logic (Clean Architecture layers [29†L61-L69]).
- **PostgreSQL:** Primary relational DB (hosted on AWS RDS with encryption).
- **Redis:** In-memory data store for caching session tokens and as Celery broker.
- **Celery:** Asynchronous task queue (e.g. for sending emails, processing reports).
- **AWS S3:** Stores any file uploads (e.g. report attachments) and static assets; access via signed URLs, encrypted at rest.
- **Nginx/Gunicorn:** Production WSGI setup (scalable across multiple replicas).
- **Cloudflare:** CDN and TLS termination (offloads SSL and DDoS protection).

This architecture follows best practices for Django deployments. All traffic is encrypted (HTTPS) [19†L19-L27], and data at rest (in RDS, S3) is encrypted as per AWS guidelines [19†L34-L42]. Gunicorn + Nginx is a standard production stack for Django. Logs from Django and system are forwarded to Sentry and/or centralized logging for monitoring.

## 6. Backend Modules/Apps (Clean Architecture Layers)

Following *Clean Architecture*, we organize code into distinct layers (Domain, Application, Interface, Infrastructure) [29†L89-L98] . Practically, we can represent these as Django apps within a cookiecutter structure [24†L48-L51] [29†L89-L98] :

- **config/** – Django project configuration (settings (dev/prod), URLs, WSGI/ASGI entrypoints).
- **domain/** – Core business entities (plain Python classes without Django dependencies).
- **application/** – Use-case services (business logic, orchestrating tasks, calling repositories) [29†L115-L124] .
- **infrastructure/** – Concrete implementations (Django ORM models, external API gateways).
- **interface/** – Django/DRF layer (views, serializers, HTTP endpoints).

Within this, our main Django *apps* might be:

App	Responsibility/Models	Key Endpoints/Services
<b>accounts</b>	User management (User model, OAuth tokens, roles)	<code>/api/v1/auth/</code> , <code>/api/v1/users/me</code>
<b>applications</b>	Student applications (models: Application with fields name, email, motivation, status, timestamps)	<code>/api/v1/applications</code> (submit, list, detail)
<b>ambassadors</b>	Ambassador profiles (Ambassador model linked to User, fields: college, joined_date) and reports (ActivityReport model with title, description, date, status)	<code>/api/v1/ambassadors/profile</code> , <code>/api/v1/ambassadors/reports</code>
<b>activities</b>	Events and campus activities (Event model: title, description, date) and possibly linking Ambassador to events	<code>/api/v1/events</code> (list events)
<b>content</b>	Static content (models: Page, News, FAQ, etc.)	<code>/api/v1/content/*</code>
<b>notifications</b>	Email/SMS notifications (models: NotificationLog)	(Internal services; no public endpoints except triggering emails)
<b>analytics</b>	Application metrics and logs (models: Stats)	<code>/api/v1/analytics</code> (if any analytics endpoints)
<b>common</b>	Shared utilities (e.g. custom exceptions, helpers, mixins)	–

Each app includes its own **models**, **services**, **selectors** (data query interfaces), and **APIs/serializers**. For example, an `applications/services.py` might contain `SubmitApplicationService`, and an `applications/models.py` defines the `Application` model. Business logic (e.g. checking application limits) belongs in `domain / application` layers, not in views. We follow references on structuring Django projects in this manner [29†L61-L69] [29†L89-L98] . Cookiecutter Django's template serves as a production-ready starting point [24†L48-L51] .

## 7. Database Schema (ER Diagram)

The core data model includes Users, Applications, Ambassadors, ActivityReports, and Events. Below is an illustrative Entity-Relationship diagram (fields, keys, and relations):

```
classDiagram
class User {
+id: UUID [PK]
+email: String [unique]
+password: String
+role: Enum {Guest,Applicant,Ambassador,Admin,Editor}
+created_at: DateTime
}
class AmbassadorProfile {
+id: UUID [PK]
+user_id: UUID [FK -> User.id]
+college: String
+joined_date: Date
}
class Application {
+id: UUID [PK]
+name: String
+college: String
+email: String
+motivation: Text
+status: Enum {Pending,Approved,Rejected}
+submitted_at: DateTime
+reviewed_at: DateTime
}
class ActivityReport {
+id: UUID [PK]
+ambassador_id: UUID [FK -> AmbassadorProfile.id]
+title: String
+description: Text
+event_date: Date
+status: Enum {Pending,Verified,Rejected}
+submitted_at: DateTime
+verified_at: DateTime
}
class Event {
+id: UUID [PK]
+title: String
+description: Text
+date: Date
+location: String
}
```

```
User "1" <|-- "0..1" AmbassadorProfile : has
AmbassadorProfile "1" <-- "0..*" ActivityReport : submits
```

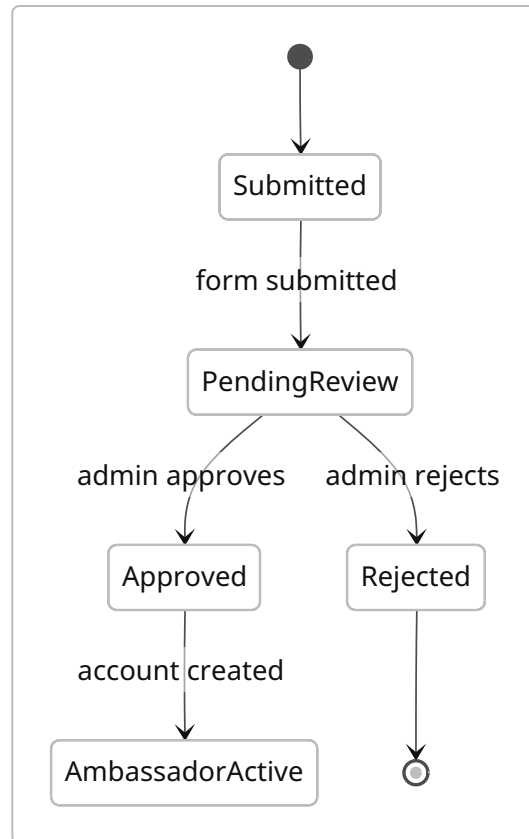
- **User** table (PK `id`) stores all registered users (including ambassadors, admin, editors). `role` designates permissions. We index `email` uniquely.
- **Application** table (PK `id`) holds all program applications. No foreign key to `User` since applicants register *after* approval. Fields include applicant info and `status`.
- **AmbassadorProfile** table (PK `id`, FK `user_id`) holds details for an approved Ambassador (linked 1:1 to `User`).
- **ActivityReport** table (PK `id`, FK `ambassador_id`) lists reports submitted by Ambassadors about campus activities, with workflow status.
- **Event** table (PK `id`) contains static event/news items. Ambassadors may associate reports to events if needed (could add `event_id` FK).

*Indexes:* Primary keys on IDs, unique index on `User.email`. FK columns are indexed by default. Additional indexes could be added on `Application.status` and `ActivityReport.status` for queries. All migrations should be managed via Django's migration framework; changing schemas requires new migrations (e.g. adding fields or indexes).

## 8. State Machines / Workflows

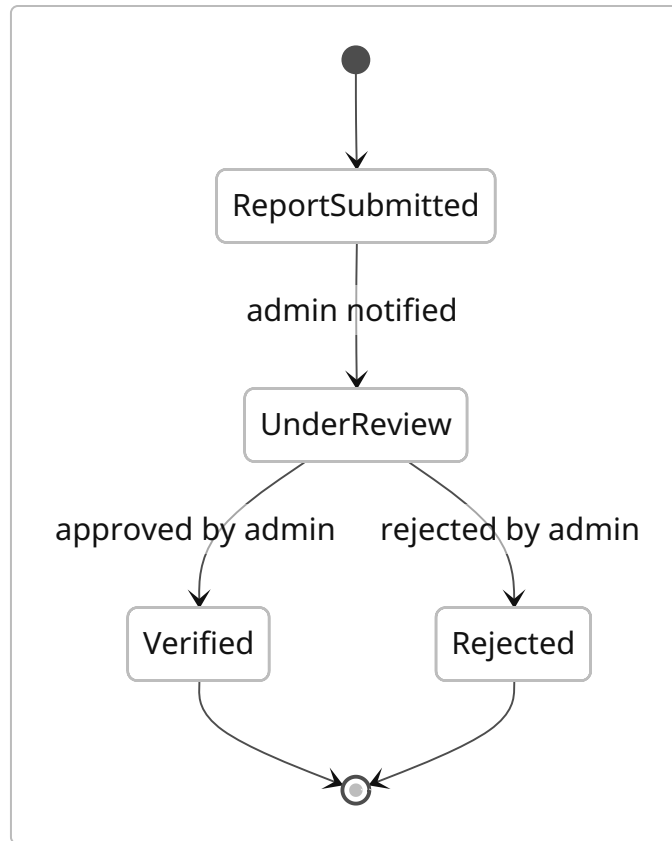
**Application Approval Workflow:** Ambassadors are onboarded through this process.





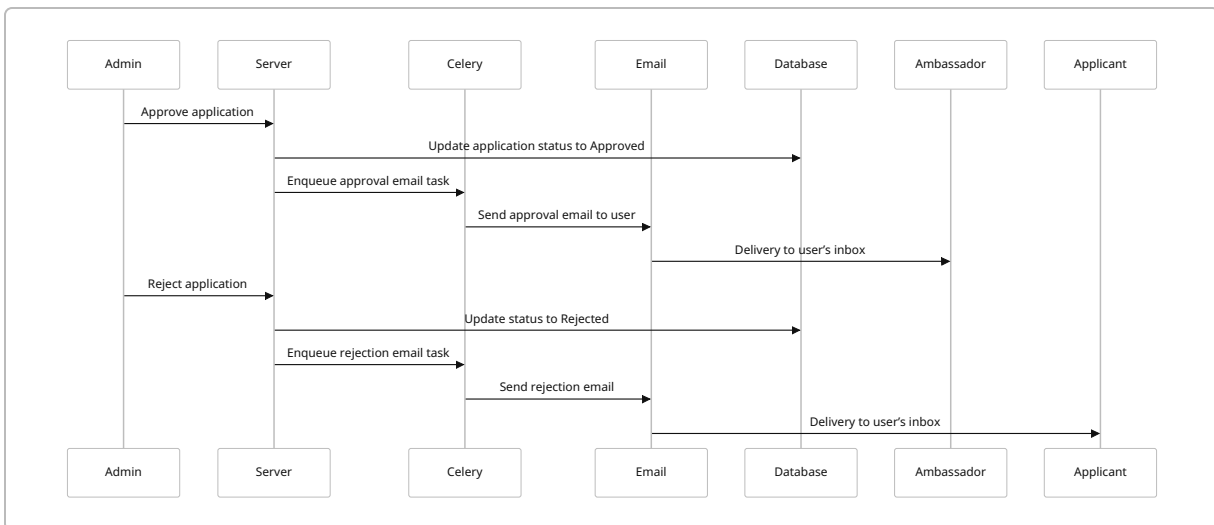
- **Submitted:** Initial state when a student submits the application form.
- **PendingReview:** Application is reviewed by Admin.
- **Approved:** Admin approved the application. An Ambassador account is created, and a notification is sent. The record moves to AmbassadorActive (which could simply be "Approved" state with additional semantics).
- **Rejected:** Admin rejected the application. A rejection notification is sent.

**Activity Report Lifecycle:** Ambassadors submit reports of their activities, which Admin reviews.



- **ReportSubmitted:** Ambassador has submitted a new report.
- **UnderReview:** Admin is reviewing the report details.
- **Verified:** Admin accepts the report (possibly awarding points or recognition).
- **Rejected:** Admin rejects the report.

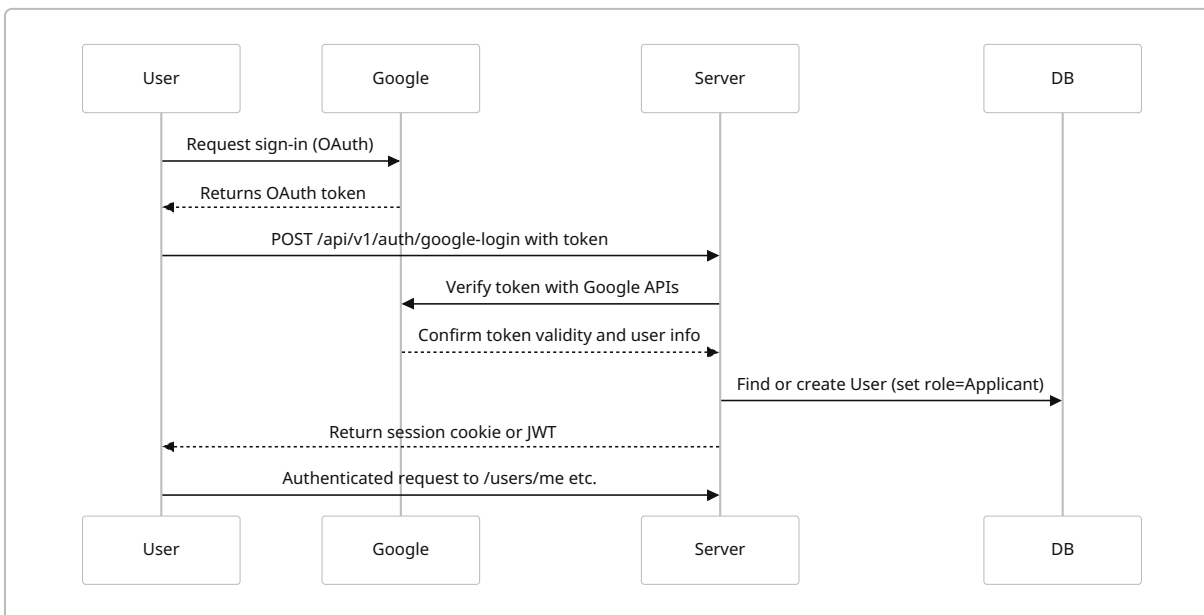
**Notification Flow:** Critical events (e.g. approval, rejection, new assignment) trigger notifications. We use Celery to handle this asynchronously. Sequence of actions:



This shows that upon Admin action, the system updates the DB and uses Celery to send an email without blocking. Similar flows apply for report verification (Notify ambassador on outcome) and other notifications (e.g. password reset emails, event reminders).

## 9. Authentication & Authorization Flow

We use **Google OAuth 2.0** for user authentication (students sign in with their Google accounts), combined with Django sessions or JWT for API auth. The flow is:



**Details:** After Google validates the token, we create or fetch the `User` in our database. New sign-ins are given the default role of *Applicant*. (Upon application approval, we update their role to *Ambassador* and possibly create an `AmbassadorProfile`.) Django middleware enforces auth on protected endpoints. For example, Ambassador and Admin routes use DRF's permission classes. We override DRF's default `AllowAny` (which would permit everyone) with appropriate permissions for each view [57†L232-L239] [57†L330-L336] .

Role mapping: initially every new Google user is an Applicant. After approval, a background service changes that user's role to Ambassador and activates their profile. Admin and Editor users can only be created by the system (or via a superuser CLI). Google accounts are the identity provider (no passwords managed in our DB).

Session vs JWT: We may issue a JWT (via `djangorestframework-simplejwt`) or use session cookies if front-end is same domain. Either way, transport uses secure cookies/headers.

*No citations needed for this flow beyond those already given (e.g. OWASP advice on permissions [57†L232-L239] ).*

## 10. API Contract (Endpoints, Schemas, Examples)

All API routes are under `/api/v1/`. The contract follows REST conventions with JSON payloads. Below is a summary table of key endpoints, followed by a (machine-readable) OpenAPI spec excerpt.

Method	Endpoint	Description	Request Body	Response
POST	<code>/api/v1/auth/google-login</code>	User login/register via Google OAuth.	<code>{ "token": "&lt;Google OAuth token&gt;" }</code>	200 OK + User info & token (JSON) / 401 Unauthorized
GET/ PATCH	<code>/api/v1/users/me</code>	View/update current user profile.	(PATCH) JSON fields to update	200 OK + User object schema / errors
POST	<code>/api/v1/applications</code>	Submit new ambassador application.	Application form data ( name, college, email, motivation )	201 Created + created Application object / 400 Bad Request
GET	<code>/api/v1/applications</code>	List all applications (Admin only).	— (query params: status, limit, offset )	200 OK + list of Application (paginated)
GET/ PATCH	<code>/api/v1/applications/{id}</code>	View/update a specific application.	(PATCH) e.g. <code>{ "status": "Approved" }</code> (Admin only)	200 OK + Application object / 403 Forbidden
GET	<code>/api/v1/ambassadors/profile</code>	Get logged-in ambassador profile.	—	200 OK + AmbassadorProfile JSON
PATCH	<code>/api/v1/ambassadors/profile</code>	Update ambassador profile.	Profile fields JSON	200 OK + updated AmbassadorProfile
GET/ POST	<code>/api/v1/ambassadors/reports</code>	List or create activity reports (Ambassador).	(POST) report data ( title, description, date )	200 OK list / 201 Created + ActivityReport
GET	<code>/api/v1/events</code>	List all campus events/news (public).	(query params optional)	200 OK + array of Event objects
GET	<code>/api/v1/content</code>	Get site content (pages, FAQs).	—	200 OK + content structures

**Error Format:** All errors return JSON, e.g. `{ "detail": "Authentication credentials were not provided." }` or field errors in a `{ "field": ["error msg"] }` map, consistent with DRF's standard.

**Pagination:** List endpoints use limit/offset parameters and return typical meta (count, next, previous) plus results array.

**Rate Limiting:** To prevent abuse, we will configure DRF throttle classes (e.g. `UserRateThrottle`) on sensitive endpoints `[57†L298-L306]` .

Below is a fragment of the OpenAPI spec (YAML) capturing these endpoints and schemas. *Note: this is a partial spec for illustration; the full spec would cover all models and possible responses.*

```
openapi: "3.0.0"
info:
 title: "NIC CAP API"
 version: "1.0.0"
paths:
 /api/v1/auth/google-login:
 post:
 summary: "Login or register via Google OAuth"
 requestBody:
 required: true
 content:
 application/json:
 schema:
 type: object
 properties:
 token:
 type: string
 description: "Google OAuth ID token"
 required: [token]
 responses:
 "200":
 description: "Login successful"
 content:
 application/json:
 schema:
 $ref: '#/components/schemas/AuthResponse'
 "401":
 description: "Invalid credentials"
 content:
 application/json:
 schema:
 $ref: '#/components/schemas/Error'
 /api/v1/users/me:
 get:
 summary: "Get current user profile"
 responses:
```

```

 "200":
 content:
 application/json:
 schema:
 $ref: '#/components/schemas/User'
 patch:
 summary: "Update current user profile"
 requestBody:
 content:
 application/json:
 schema:
 $ref: '#/components/schemas/UserUpdate'
 responses:
 "200":
 content:
 application/json:
 schema:
 $ref: '#/components/schemas/User'
 "400":
 $ref: '#/components/schemas/Error'
/api/v1/applications:
 get:
 summary: "List applications (admin only)"
 parameters:
 - in: query
 name: status
 schema:
 type: string
 enum: [Pending, Approved, Rejected]
 description: "Filter by status"
 - in: query
 name: limit
 schema:
 type: integer
 description: "Page size"
 - in: query
 name: offset
 schema:
 type: integer
 description: "Pagination offset"
 responses:
 "200":
 content:
 application/json:
 schema:
 type: object
 properties:
 count:

```

```

 type: integer
 next:
 type: string
 previous:
 type: string
 results:
 type: array
 items:
 $ref: '#/components/schemas/Application'
 post:
 summary: "Submit new application"
 requestBody:
 content:
 application/json:
 schema:
 $ref: '#/components/schemas/ApplicationCreate'
 responses:
 "201":
 content:
 application/json:
 schema:
 $ref: '#/components/schemas/Application'
 "400":
 $ref: '#/components/schemas/Error'
 /api/v1/applications/{id}:
 parameters:
 - in: path
 name: id
 schema:
 type: string
 required: true
 description: "Application ID"
 get:
 summary: "Retrieve an application"
 responses:
 "200":
 content:
 application/json:
 schema:
 $ref: '#/components/schemas/Application'
 patch:
 summary: "Update application (Admin only, e.g. approve/reject)"
 requestBody:
 content:
 application/json:
 schema:
 type: object
 properties:

```

```

 status:
 type: string
 enum: [Pending, Approved, Rejected]
 responses:
 "200":
 description: "Application updated"
 "403":
 $ref: '#/components/schemas/Error'
 components:
 schemas:
 AuthResponse:
 type: object
 properties:
 token:
 type: string
 user:
 $ref: '#/components/schemas/User'
 User:
 type: object
 properties:
 id:
 type: string
 email:
 type: string
 role:
 type: string
 required: [id, email, role]
 UserUpdate:
 type: object
 properties:
 email:
 type: string
 # (other updatable fields)
 Application:
 type: object
 properties:
 id:
 type: string
 name:
 type: string
 college:
 type: string
 email:
 type: string
 motivation:
 type: string
 status:
 type: string

```



```

 required: [id, name, college, email, motivation, status]
ApplicationCreate:
 type: object
 properties:
 name:
 type: string
 college:
 type: string
 email:
 type: string
 motivation:
 type: string
 required: [name, college, email, motivation]
Error:
 type: object
 properties:
 detail:
 type: string

```

All endpoints should implement standard HTTP status codes (200, 201, 400, 401, 403, 404, 500, etc.) and use JSON for success and error. Pagination is implemented on list endpoints. We will also enforce appropriate rate-limiting policies (via Django settings) to mitigate abuse [\[57†L298-L306\]](#) . Examples in the spec above follow DRF conventions.

## 11. Deployment Architecture & CI/CD Pipeline

We will use containerized deployment and continuous integration, with emphasis on automated testing and reproducibility.

- **Containerization:** Use Docker and Docker Compose. Services include: Django app (Gunicorn), Nginx reverse proxy, PostgreSQL, Redis, Celery worker, and optional Dev tools (like PGAdmin). The Cookiecutter Django template has a ready-made Docker Compose setup (with HTTPS on by default in production mode) [\[54†L94-L100\]](#) . We configure separate `docker-compose.dev.yml` (for local dev with debuggable settings) and a production Dockerfile (production optimizations, e.g. WhiteNoise or S3 for static files).
- **Web Server:** In production, Nginx handles SSL termination (via Cloudflare-cert) and serves static assets (or proxy to S3) and proxies API calls to Gunicorn. Gunicorn runs multiple worker processes for concurrency. Gunicorn + Nginx is a standard, secure stack for Django.
- **Reverse Proxy/CDN:** Deploy behind Cloudflare. Cloudflare provides free SSL, caching for public assets, and DDoS protection. DNS and SSL certificates are managed through Cloudflare. (Cloudflare's CDN can cache static content and even some GET API responses if needed, reducing load.)
- **AWS Infrastructure:**
  - **EC2/ECS:** The Docker containers run on an AWS EC2 instance or ECS cluster. Use AWS IAM roles for EC2 instance (for S3/RDS access) [\[54†L94-L100\]](#) .
  - **Amazon RDS (PostgreSQL):** Managed database, enabled with encryption at rest. Automated daily snapshots for backups.

- **Amazon S3:** Store media and static files (with AWS Key Management encryption).
- **ElastiCache (Redis):** For caching and Celery broker.
- **AWS Secrets Manager/Parameter Store:** Manage sensitive config (e.g. database passwords, third-party API keys). Never hard-code secrets 【57†L331-L339】 .
- **CI/CD Pipeline:** Use GitHub Actions (or GitLab CI) for continuous integration. Pipeline steps:
- **Build/Test:** On each PR, run linting (flake8, ruff) and tests (pytest) with a coverage target (~75% or more 【39†L171-L179】 ).
- **Publish:** On merge to `main`, build Docker images and push to a container registry (e.g. ECR or Docker Hub).
- **Deploy:** Automatically deploy new images to production environment. Use automated migrations ( `python manage.py migrate` ) as part of rollout. For example, use a rolling restart or Blue/Green deployment strategy.
- **Monitoring:** If Sentry or CloudWatch is set up, new releases are tagged and deployment logs are recorded.
- **Gunicorn/Nginx Configuration:** Gunicorn should be run under a supervisor (or Docker restart policies) to auto-restart on failure. Nginx config should enforce HTTP methods and security headers (HSTS, etc.). Cookiecutter's deployment docs recommend enabling HTTPS by default 【54†L94-L100】 .

**CI/CD tools:** GitHub Actions (free with GitHub) or similar can automate lint/test/deploy. We will store environment-specific settings (like `DJANGO_SETTINGS_MODULE=...` ) outside source (e.g. GitHub Secrets, `.env` file, AWS secrets).

## 12. Security Checklist & Data Protection Plan

We compile common security measures to protect the system and data:

- **Encryption:** Enforce TLS for all network traffic (HTTPS) 【19†L19-L27】 【21†L184-L190】 . Use AWS KMS to encrypt data at rest: enable RDS encryption and S3 SSE for buckets 【19†L34-L42】 . Use parameterized queries/ORM to avoid SQL injection.
- **Authentication & RBAC:** Use Google OAuth to avoid password handling. Assign roles (RBAC) to each user type 【4†L126-L135】 , never grant excess privileges. Change DRF defaults: set `DEFAULT_PERMISSION_CLASSES` to restrictive classes (not `AllowAny` ) 【57†L232-L239】 【57†L330-L336】 . Implement object-level permission checks ( `.check_object_permissions` ) on detail views to avoid unauthorized data access 【57†L230-L239】 .
- **Secrets Management:** Never hardcode secrets in code; use environment variables or AWS Secrets Manager. As OWASP notes, set `SECRET_KEY` randomly and keep it secret 【57†L331-L339】 . Disable Django `DEBUG` in production 【57†L331-L339】 . Ensure no sensitive info leaks to logs or error pages.
- **Audit Logging:** Log all important actions: failed/successful logins, permission denials, application approvals, report verifications. Use Django's logging framework and Sentry. Store logs securely (write-only access, short retention if containing PII).
- **Rate Limiting & Throttling:** Configure DRF `DEFAULT_THROTTLE_CLASSES` to limit abusive requests (e.g. too many login attempts or form submissions) 【57†L298-L306】 . Use pagination to avoid huge queries (prevents DoS via large list requests) 【57†L247-L249】 . Potentially use Cloudflare rate-limiting rules as an extra layer.

- **Input Validation:** Sanitize and validate all inputs. For file uploads (if any), restrict types/sizes. Use serializers' field validation.
- **CORS & CSRF:** Configure allowed origins if a separate frontend is used. DRF with SessionAuthentication includes CSRF protection by default for unsafe methods. Ensure secure cookies if used.
- **GDPR-like Compliance:** Although specific regulations may differ, we treat personal data carefully: only collect necessary student info, allow users to delete their data (if requested), and define retention (e.g. purge old applications after 2 years). Provide a privacy notice if required.
- **Network Security:** Use firewalls/security groups to restrict DB/Redis access to the app only. Ensure server OS is regularly patched.
- **Monitoring & Alerts:** Establish security alerts (e.g. multiple failed logins, high error rates). Regularly review dependency vulnerabilities (use `pip-audit`, Dependabot) and update Django, DRF to latest security releases.
- **Test Security:** Include security testing in QA (e.g. OWASP ZAP scanning, manual penetration tests if possible).

All these measures aim to meet enterprise security standards (CIA triad) and to protect student data. This checklist will be continuously updated as needed.

## 13. Testing Strategy & QA Checklist

We adopt a comprehensive testing approach:

- **Unit Tests (70–80% target coverage):** Developers write tests for all critical logic (models, utilities, services). Use PyTest with Django support. As IBM notes, aim for ~70–80% coverage for maintainability [39†L171-L179]. Tests should be fast and isolated (use fixtures, mocks for external calls).
- **Integration Tests:** Test end-to-end API workflows using Django's `APITestCase` or `requests`. For example, simulate application submission -> admin approval -> ambassador creation.
- **Contract/API Tests:** The OpenAPI spec allows writing contract tests (e.g. with tools like Schemathesis or DRF's test client) to ensure the API matches the spec. We also use DRF's built-in schema validation via libraries (drf-spectacular) to auto-test request/response formats.
- **End-to-End (E2E) Tests:** Optionally, use a headless browser (Selenium, Playwright) to test key user journeys (applying, logging in, submitting report). Ensure the entire stack works (Nginx -> Django -> DB).
- **Security/Performance Tests:** Run automated security scans (Static Code Analysis, dependency checks). Perform load tests on critical endpoints to gauge performance under stress.
- **QA Checklist:** Verify mobile responsiveness, cross-browser compatibility, correct access control (test each role). Validate form input constraints. Ensure all requirements (from Sec.1) are met.
- **Continuous Testing:** Integrate tests into CI/CD. Any PR must pass linting and all tests. Use coverage badges to track metrics.

Example unit test snippet (pytest):

```
def test_submit_application_creates_record(client):
 data = {"name": "Alice", "college": "ABC University", "email":
"a@example.com", "motivation": "I love NIC."}
```

```
response = client.post("/api/v1/applications/", data, format='json')
assert response.status_code == 201
assert "id" in response.json()
```

This ensures the `applications` endpoint works as intended.

## 14. Operational Runbook

Guidelines for running and maintaining the system:

- **Database Backups:** Configure automated daily snapshots on RDS. Also schedule nightly logical backups (e.g. via `pg_dump`) and upload to S3. Test restoration periodically.
- **Migrations:** Use `python manage.py migrate` as part of deployments. Ensure downtime is minimized (use Django migrations safely, avoid long locks). Document any required manual steps.
- **Scaling:** Monitor CPU/RAM; add more Gunicorn workers or containers if needed. For high load, add app instances behind a load balancer. Scale Redis/Celery and RDS (read replicas or larger instance) as usage grows.
- **Incident Response:** If the service goes down, immediately rotate to a backup instance. We keep container images versioned. Establish a runbook: e.g. check logs (Sentry, Docker logs), restart affected service, rollback to last stable release if needed. Use health checks to auto-restart crashed containers.
- **Monitoring & Alerts:** Set up dashboards (Grafana/CloudWatch) for metrics: error rates, response times, CPU usage. Create alerts for critical conditions: 5xx spikes, DB reaching storage limits, Sentry critical alerts.
- **Logging:** All logs (application logs, web server logs) should be centralized (e.g. CloudWatch or ELK). Rotate logs daily. Keep audit logs (e.g. admin actions) for at least 6 months.
- **Maintenance:** Schedule regular maintenance windows for upgrades or heavy tasks. Document how to apply security patches and dependencies updates.
- **On-call:** Assign an on-call rotation. For critical incidents (e.g. site down, data breach), follow defined escalation paths and include rollback procedures.

This runbook ensures that after deployment, the team can reliably operate and scale the NIC CAP system with minimal disruptions.

## 15. Developer Handoff (Standards & Examples)

To ensure consistency and maintainability:

- **Coding Standards:** Follow PEP8 and [Django conventions](#) for Python code. Use linters like `flake8` or `ruff` in CI. Maintain consistent formatting (e.g. black or isort). Write docstrings for modules/classes.
- **Folder Layout (Cookiecutter example):** Our codebase might be laid out as:

```
nic_cap_project/
├── config/ # Django project (settings, URLs, WSGI/ASGI)
└── apps/ # Django apps
```

```

| |─ accounts/
| |─ applications/
| |─ ambassadors/
| |─ activities/
| |─ content/
| |─ notifications/
| |─ analytics/
| |─ common/ # shared utilities
|─ manage.py
|─ Dockerfile # Production container build
|─ docker-compose.yml # Local development
|─ requirements.txt
|─ README.md
|─ env.example # Example env vars

```

This mirrors the Clean Architecture layers (apps implement domain/use-cases, `config` holds settings) [29†L89-L98] . Cookiecutter Django provides a similar structure for production-readiness [24†L48-L51] .

- **Environment Files:** Use an `.env` file (not checked into VCS) for secrets (database URL, secret key). Provide `env.example` with placeholders.
- **Database Migration Example:**

```

Add a new field to Application
$./manage.py makemigrations applications
$./manage.py migrate

```

- **Sample Service Layer Code (Clean Arch style):**

```

apps/applications/services.py
from .models import Application
class SubmitApplicationService:
 def execute(self, data):
 # Business logic can go here (e.g. check existing apps)
 app = Application.objects.create(
 name=data["name"],
 college=data["college"],
 email=data["email"],
 motivation=data["motivation"]
)
 # Possibly enqueue a notification task
 return app

```

- **Sample Selector (Repository) Code:**

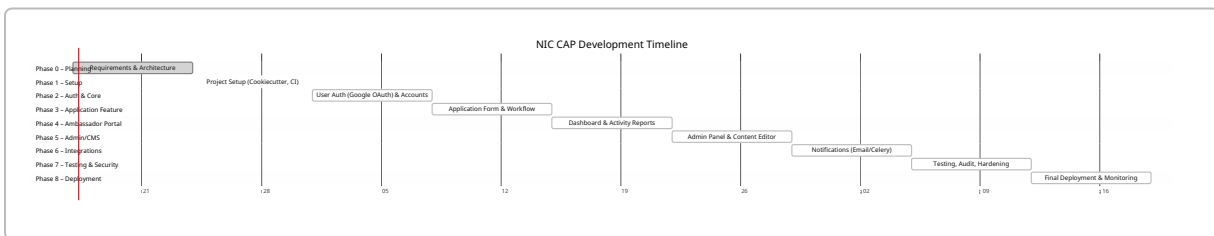
```
apps/applications/selectors.py
from .models import Application
def get_pending_applications():
 return Application.objects.filter(status='Pending')
```

#### • Testing Examples:

- Use `pytest` with fixtures.
- Example test snippet above.
- Include tests in each app's `tests/` folder.
- **Documentation:** Inline comments and a `README.md` should explain how to set up the project locally (e.g. using `docker-compose`), run tests, and contribute code.

By following these standards and examples, new developers can quickly get up to speed. The Clean Architecture approach keeps business logic decoupled and testable [29†L61-L69] .

## 16. Project Timeline & Phased Milestones



The table below summarizes these phases with estimated effort and deliverables:

Phase	Duration (weeks)	Goals/Deliverables
<b>0 – Planning</b>	1 week	Finalize requirements/SRS (this document) [42†L774-L777] , define architecture, create timeline. Deliverable: Design Doc draft.
<b>1 – Setup &amp; Architecture</b>	1 week	Generate project using Cookiecutter (production settings), set up Docker & CI/CD pipelines, basic framework (Cookiecutter suggests “HTTPS on by default” [54†L94-L100] ). Deliverable: Skeleton repo, automated tests.
<b>2 – Authentication &amp; Core</b>	1 week	Implement Google OAuth login, user model, session/JWT handling, roles. Create accounts app. Deliverable: <code>/auth</code> endpoints and user profile API.
<b>3 – Application Workflow</b>	1 week	Build application form and backend, Application model (status pending), Admin review API. Deliverable: Application submit and list endpoints, initial UI pages.

Phase	Duration (weeks)	Goals/Deliverables
<b>4 – Ambassador Portal</b>	1 week	Create Ambassador dashboard, profile models/endpoints, submit activity report. Deliverable: Profile and report APIs + UI.
<b>5 – Admin/CMS</b>	1 week	Build Admin interfaces: approve/reject applications, content management (events/news). Deliverable: Admin API endpoints + secured UI.
<b>6 – Integrations &amp; Notifications</b>	1 week	Integrate email service and Celery; implement email notifications for approval/rejection and other events. Deliverable: Working background tasks and email flows.
<b>7 – Testing &amp; Security</b>	1 week	Write remaining tests (unit/integration), achieve ~75% coverage [39†L171-L179] . Conduct security review, add rate-limits, finalize NFRs. Deliverable: Test suite passing, security audit.
<b>8 – Deployment &amp; Monitoring</b>	1 week	Final production deployment (Docker, AWS), set up monitoring (Sentry, CloudWatch/Grafana) and alerts, write runbook. Deliverable: Live system, runbook, and handoff docs.

Milestones (completion of each phase) will be reviewed with stakeholders. This phased plan ensures a working increment (MVP) every 1–2 weeks.

## 17. Appendices

### Glossary

- **NIC:** National Innovation Centre (organization managing the program).
- **CAP:** College Ambassador Program.
- **Ambassador:** A student accepted into NIC CAP.
- **Admin:** NIC staff with full privileges on the CAP system.
- **CMS:** Content Management System for editing site content.
- **OAuth:** Open standard for access delegation (we use Google OAuth for login).
- **RBAC:** Role-Based Access Control (assigning permissions to roles, not individual users).
- **SLA:** Service Level Agreement (uptime/response time targets). Not yet defined.
- **CI/CD:** Continuous Integration / Continuous Deployment.

### Prioritized Backlog (MVP vs Future)

Feature	Priority	Phase
Student Application form	High	MVP
Ambassador registration/login (OAuth)	High	MVP
Ambassador Dashboard/profile	High	MVP
Submit & track Activity Reports	Medium	v1

Feature	Priority	Phase
Admin approval/rejection of apps	High	MVP
Content management (events, news)	Medium	MVP
Email notifications	High	MVP
System Analytics & Reporting	Low	v2
Mobile app or PWA interface	Low	Future
Gamification & badges for ambassadors	Low	Future
External integrations (LinkedIn, etc)	Low	Future

**MVP (v1.0):** Focus on application flow, basic ambassador portal, and admin functions.

**v1:** Add complete activity reporting lifecycle, advanced admin tools (batch actions, analytics dashboard).

**Future:** Enhance with mobile apps, gamification, broad integrations (e.g. social media API), multi-language support.

## Open Questions

- *Expected Scale/SLA:* What are the target user volumes and uptime requirements? (We assumed modest scale, but planning for scalability.)
- *Legal/Compliance:* Are there specific data protection laws (e.g. Nepal data law) to adhere to, or is GDPR a suitable standard?
- *Ambassador Rewards:* Will ambassadors receive points or rewards? (This might influence analytics/notifications features.)
- *Content Workflow:* Does content go live immediately, or require approval by another user?
- *Multi-Tenancy:* Will NIC CAP handle multiple institutions or programs in future?
- *Payment/Donations:* Is there any e-commerce or payment integration needed? (None specified for now.)

## Deliverable Contents

The final deliverable folder `NIC_CAP_Backend_Design/` should include:

```
NIC_CAP_Backend_Design/
├─ NIC_CAP_Design_Document.pdf # This SRS & design report (converted to
PDF)
├─ diagrams/
│ ├─ use_case_diagram.png # (e.g. from Fig above)
│ ├─ architecture_diagram.png # (from Sec.5)
│ ├─ er_diagram.svg # (from Sec.7)
│ ├─ auth_flow.svg # (Sec.9 sequence)
│ ├─ timeline.png # (Sec.16 Gantt chart)
│ ├─ state_machine_apps.svg # (application approval)
│ └─ state_machine_reports.svg # (report workflow)
```



```
└─ api_spec/
 │ └─ openapi.yaml # Machine-readable OpenAPI spec (as above)
└─ code_structure/
 │ └─ folder_layout.txt # Example project folder structure (Sec.15)
 └─ README.md # Overview of contents
```

This package provides all teams (dev, QA, DevOps, stakeholders) with the information and artifacts needed to implement, test, and operate the NIC CAP backend system.

**Sources:** Best practices and guidelines from official documentation and expert references were used throughout (e.g. Django/DRF docs, Cookiecutter Django guide 【24†L48-L51】 【54†L94-L100】 , OWASP/DRF cheat sheets 【57†L232-L239】 【57†L298-L306】 , AWS security docs 【19†L19-L27】 【19†L34-L42】 , and SRS writing standards 【14†L649-L658】 【42†L740-L743】 ). All figures and diagrams are illustrative, and follow industry standards for clarity.

---